

Chapter 6 : Testing

Aviral Janveja

1 Testing

We have written many programs in this course, mostly to demonstrate a particular concept, however when developing software in a professional setting, it is imperative that we provide well functioning, error free and robust code.

This is where testing and debugging comes in. It helps us make sure, that our code satisfies all the requirements, runs error free and is able to handle edge-cases as well. The first step in this process, is to set yourself up for easy testing and debugging by writing clean, modular and well-documented code.

1.1 Testing Approaches

Software testing can be majorly classified into two high level approaches :

- **Black Box Testing** : Black box testing is a testing technique in which the internal workings of the code are not known to the test team. They only focus on the input and output of the program.
- **White Box Testing** : White box testing is a testing technique in which the test team has knowledge of the internal workings of the code and can test individual code snippets and functions.

1.2 Testing Classes

There are three general classes of testing that you can perform :

- **Unit Testing** : Unit tests focus on validating small testable parts of an application. In context of Python, unit tests make sure that individual functions run according to their specifications.
- **Integration Testing** : Integration testing examines the interaction between different components, modules or systems to ensure that they work together as intended.
- **Regression Testing** : Regression testing aims to ensure that recent changes or enhancements to an application do not introduce new bugs or break existing functionality.

1.3 Common Error Messages

Some common error messages and their meanings :

1. **SyntaxError** : This error occurs when the Python interpreter encounters code that doesn't follow the syntax rules. It could be a missing parenthesis, a misplaced colon, or some other syntax-related issue.
2. **NameError** : These types of errors can result from attempting to use a variable or function that hasn't been defined. Make sure to check for typos in variable or function names, and make sure they are defined before use.
3. **IndexError** : This type of error happens when you are trying to access an index that doesn't exist. To fix it, make sure that the index being used is within the valid range.
4. **TypeError** : This is a common exception in Python that occurs when an operation or function is applied to an object of an inappropriate type. For example, attempting to add a string and an integer.
5. **ValueError** : It is a type of error that occurs when a function receives an argument of the correct type but an incorrect value. For example, when trying to convert a string like "abc" to an integer.

2 Debugging

Now that we understand some common error types, let's explore the tools and techniques that can help debug Python code efficiently.

2.1 Print Statements

It is essential to understand how your code is executing and the values of variables at different points in the program. By strategically placing print statements at different check-points within your code, you can see the order in which different sections of your code are executing. This helps us understand the control-flow and pinpoint where the program might be deviating from the expected path.

While print statements are often the quickest and most straightforward way to get a glimpse into a program's execution steps, especially during initial development, they can be cumbersome to manage and may not be appropriate for production-grade code.

2.2 Logging

Logging is like writing notes while your program runs. Instead of just printing things to the screen, you write them to a log. It helps you keep track of what your program is doing, especially when things go wrong.

Logging can be configured to write messages to both the console and a file named "example.log". The "format" parameter can be used to customize the appearance of log messages, including the timestamp, log-level, and the actual log message.

Further, Python development environments many a times provide inbuilt logging and debugging tools that can help make the debugging process easier. It is possible to set the log-level based on severity and customize the log messages as well.

3 Exception Handling

An error is also called an exception, as it is an exception to what the programmer expected. Common exception handling statements include :

3.1 try-except

The **try-except** statement is a way to handle exceptions in Python. It prevents your program from crashing abruptly, allowing you to handle errors safely and log relevant information. For example :

```
try:
    a = int(input("Enter a number: "))
    b = int(input("Enter another number : "))
    print("Sum = ", a + b)
    print("Division = ", a / b)

except:
    print("Invalid Input!")
```

The **try** block contains the code that might raise an exception. The **except** block contains the code that will be executed, if an exception occurs in the try block.

The above is a general except statement that can be used to catch any exception. You can also catch specific exceptions by specifying separate except clauses for different types of errors.

3.2 finally

We can also add a **finally** block that is always executed, regardless of whether an exception happens. For example :

```
try:
    x > 3

except:
    print("Something went wrong")

finally:
    print("The try-except block is finished")
```

It is used for executing clean-up code that should be run no matter what else happens, for example closing a file.

3.3 raise

It is also possible to raise our own exception using the **raise** keyword :

```
x = "hello"

if type(x) is not int:
    raise TypeError("x should be an integer")
```

3.4 assert

The **assert** keyword lets you test if a condition in your code returns True. If not, the program will raise an `AssertionError`. You can also add a message to be displayed :

```
x = "Welcome"

#if the condition returns False,
# AssertionError is raised:
assert x == "Hello", "x should be Hello"
```

Assert is useful in checking if the pre-conditions and post-conditions on your functions are exactly as you want them to be. So, as soon as an assert becomes false, the function is going to terminate immediately. This is useful as it prevents the program from propagating bad values forward. Thus, helping in spotting bugs as soon as they are introduced and where they are introduced.